

Unidad 12: Despliegue y CI/CD

Post-Contenido 1 — Laboratorio: Contenedorizar la Aplicación Spring Boot y
Desplegar en Railway

Sergio Andrés Moreno Sánchez

02230131001

Jonathan Rolando Rey Castillo

Universidad de Santander

UDES – Campus Cúcuta

Cúcuta, Norte de Santander

2026

1. Información General

- URL pública de la aplicación: <https://moreno-post1-u12production.up.railway.app/cursos>
- Repositorio GitHub: <https://github.com/SergioMoreno09dev/moreno-post1-u12.git>
- Tecnologías utilizadas: Spring Boot 4.0.5 · Docker · PostgreSQL 16 · Railway · Maven

2. Parte 1 — Dockerfile Multi-Stage

Se creó un Dockerfile en la raíz del proyecto con dos etapas bien diferenciadas.

Etapas 1 (builder): usa la imagen eclipse-temurin:21-jdk-alpine que incluye el JDK completo y Maven. Se copia primero el pom.xml para aprovechar la caché de capas de Docker, si las dependencias no cambian entre builds, Maven no las descarga nuevamente. Luego se copia el código fuente y se compila con `./mvnw clean package -DskipTests`.

Etapas 2 (producción): usa eclipse-temurin:21-jre-alpine, que solo contiene el JRE sin herramientas de desarrollo, reduciendo el tamaño final de la imagen a menos de 300 MB. Se crea un usuario no root llamado spring como buena práctica de seguridad en contenedores. Solo se copia el JAR generado en la etapa anterior.

Se creó además el archivo `.dockerignore` excluyendo `target/`, `.git/`, `.idea/` y otros archivos innecesarios para reducir el contexto de construcción.

Checkpoint 1 verificado: `docker build -t mi-app:local .` completó sin errores y `docker images` mostró la imagen con tamaño inferior a 300 MB.

3. Parte 2 — Perfiles y Docker Compose

Configuración de perfil de producción:

Se creó el archivo `application-prod.properties` en `src/main/resources/` con todas las propiedades de conexión configuradas mediante variables de entorno, sin valores hardcoded. Se especificó el driver `org.postgresql.Driver` y el dialecto `PostgreSQLDialect` para que Hibernate reconozca la base de datos PostgreSQL correctamente.

Docker Compose:

Se creó docker-compose.yml orquestando dos servicios. El servicio db usa la imagen postgres:16-alpine con un healthcheck configurado con pg_isready para verificar que PostgreSQL esté listo antes de que la aplicación intente conectarse. El servicio app depende del servicio db con la condición service_healthy, activando el perfil prod mediante la variable SPRING_PROFILES_ACTIVE=prod.

Checkpoint 2 verificado: docker compose up -d --build levantó ambos servicios correctamente. docker compose ps mostró app y db en estado Up/healthy. El endpoint http://localhost:8080/actuador/health respondió {"status":"UP"} y las páginas /cursos y /cursos/nuevo cargaron correctamente desde el navegador.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\finea\Downloads\Moreno-post1-u12> docker compose ps
time="2026-05-16T16:20:50-05:00" level=warning msg="C:\\Users\\finea\\Downloads\\Moreno-post1-u12\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"

```

NAME	SERVICE	CREATED	IMAGE	STATUS	COMMAND	PORTS
moreno-post1-u12-app-1	app	4 minutes ago	moreno-post1-u12-app	Up 4 minutes	"java -jar app.jar"	0.0.0.0:8080->8080/tcp
moreno-post1-u12-db-1	db	4 minutes ago	postgres:16-alpine	Up 4 minutes (healthy)	"docker-entrypoint.s..."	5432/tcp

```
PS C:\Users\finea\Downloads\Moreno-post1-u12> |
```

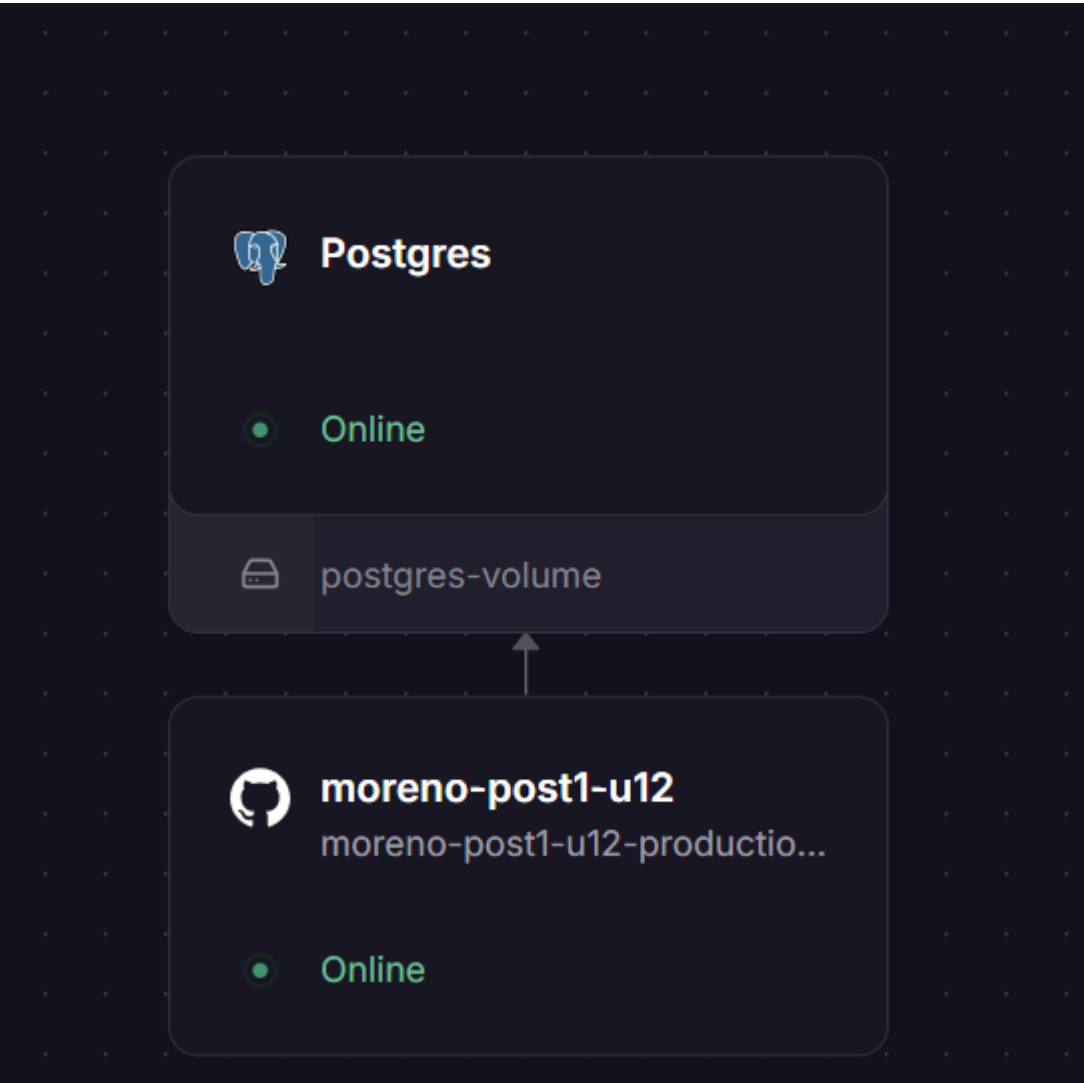
4. Parte 3 — Despliegue en Railway

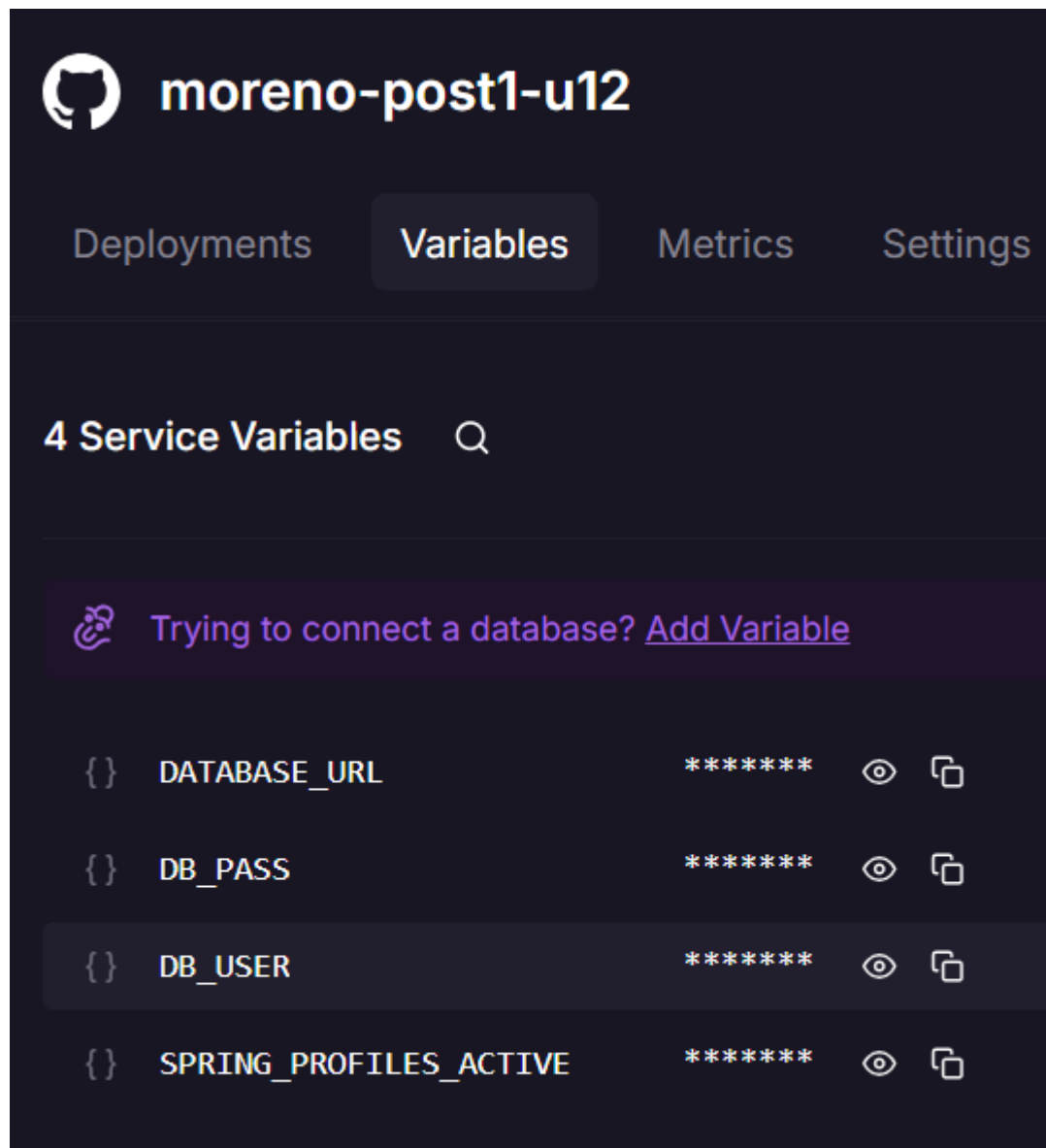
Pasos realizados:

Se inició sesión en railway.app con la cuenta de GitHub y se creó un nuevo proyecto seleccionando "Deploy from GitHub repo", conectando el repositorio Moreno-post1-u12. Railway detectó automáticamente el Dockerfile y ejecutó el primer build. Desde el panel del proyecto se agregó un servicio de base de datos PostgreSQL usando "+ New → Database → Add PostgreSQL". Railway aprovisionó la base de datos y expuso sus variables de conexión internamente.

Variables de entorno configuradas en Railway:

Variable	Valor
SPRING_PROFILES_ACTIVE	prod
DATABASE_URL	jdbc:postgresql://\${Postgres.PGHOST}:\${Postgres.PGPORT}/\${Postgres.PGDATABASE}
DB_USER	\${Postgres.PGUSER}
DB_PASS	\${Postgres.PGPASSWORD}





Checkpoint 3 verificado: Los siguientes endpoints respondieron correctamente desde la URL pública de Railway:

Endpoint	Resultado
/actuator/health	{"status":"UP"} con PostgreSQL conectado
/cursos	Página de lista de cursos cargada correctamente
/cursos/nuevo	Formulario de nuevo curso cargado correctamente

Impresión con sangría ☒

```
{
  "components": {
    "db": {
      "details": {
        "database": "PostgreSQL",
        "validationQuery": "isValid()"
      },
      "status": "UP"
    },
    "diskSpace": {
      "details": {
        "total": 1081101176832,
        "free": 1022076710912,
        "threshold": 10485760,
        "path": "/app/.",
        "exists": true
      },
      "status": "UP"
    },
    "livenessState": {
      "status": "UP"
    },
    "ping": {
      "status": "UP"
    },
    "readinessState": {
      "status": "UP"
    },
    "ssl": {
      "details": {
        "expiringChains": [],
        "invalidChains": [],
        "validChains": []
      },
      "status": "UP"
    }
  },
  "groups": [
    "liveness",
    "readiness"
  ],
  "status": "UP"
}
```



Nuevo Curso

Nombre:

Créditos:

[Volver a la lista](#)

5. Conclusiones

Se contenedorizó exitosamente la aplicación Spring Boot usando un Dockerfile multi-stage, logrando separar el ambiente de compilación del ambiente de ejecución y reduciendo el tamaño de la imagen final. La orquestación local con Docker Compose permitió simular el ambiente de producción con PostgreSQL antes del despliegue en la nube. El despliegue en Railway demostró cómo una plataforma PaaS puede detectar automáticamente el Dockerfile, construir la imagen y aprovisionar servicios de base de datos, simplificando significativamente el proceso de CI/CD. El principal desafío fue el formato de la URL de conexión a PostgreSQL, que Railway entrega en formato postgresql:// mientras que Spring Boot requiere el prefijo jdbc:postgresql://, problema que se resolvió construyendo la URL manualmente con las variables individuales de Railway.